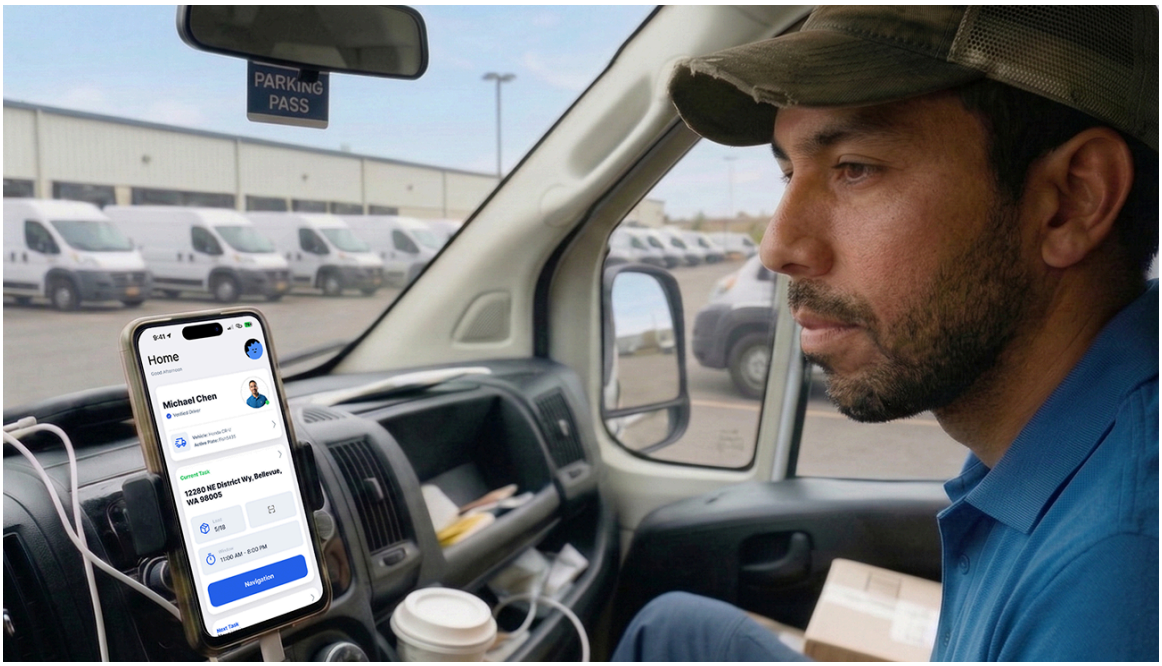
Fleeton System User Manual

Version:	2.0
Team Members:	Jingli Qin, Haichao Xing, Zeng Zhang
Instructors:	Luyao Liu, Mark Licata
Industrial Sponsor:	T-Mobile
Submission Date:	March 2026

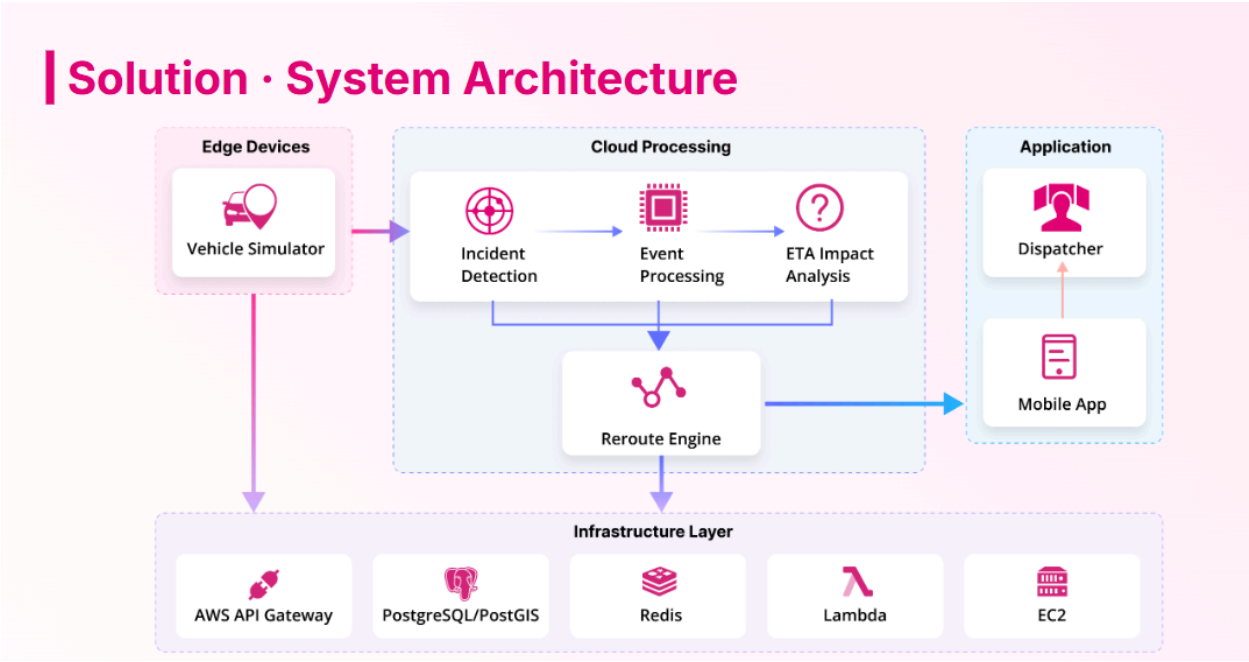


1. Product Overview

Fleeton is a real-time delivery rerouting system designed to improve reliability in last-mile logistics. Traditional delivery routing systems generate a static route before the driver departs, but real-world conditions such as traffic congestion, accidents, or unexpected delays often cause the planned sequence to become inefficient.

Fleeton bridges the gap between route planning and real-world execution by continuously monitoring vehicle telemetry data and detecting incidents that may impact delivery performance. When the system detects a significant delay risk, it dynamically recalculates the optimal delivery route and provides updated instructions to the driver.

The system consists of a backend cloud service responsible for telemetry ingestion, event processing, ETA analysis, and route optimization, as well as a driver-facing frontend application that visualizes routes, alerts drivers to incidents, and allows drivers to accept optimized routes.



Fleeton is designed for:

- delivery drivers
- fleet operators
- logistics companies managing last-mile delivery

Primary features include:

- real-time vehicle telemetry monitoring
- incident detection based on driving conditions
- ETA impact analysis
- dynamic route resequencing
- driver-facing map interface with rerouting alerts

2. System Requirements

2.1 Backend Requirements

Operating System:

- Linux or macOS recommended

Software Dependencies:

- Python 3.10+
- PostgreSQL with PostGIS
- Redis
- OSRM routing engine

Cloud Services:

- AWS Lambda
- AWS API Gateway

Hardware Requirements:

- Standard development machine capable of running Python services and PostgreSQL

Internet Requirements:

- Internet connection required for telemetry ingestion and routing requests

2.2 Frontend Requirements

Supported Platforms:

- Modern web browsers (Google Chrome, Microsoft Edge, Safari)

Software Dependencies:

- Node.js (version 18 or higher)
- npm package manager

Hardware Requirements:

- Laptop or desktop computer capable of running modern web applications

Internet Requirements:

- Internet connection required to communicate with backend APIs and receive route updates

External Services:

- Azure Maps API key required for map visualization

4. Installation and Setup

Backend Installation

Step 1: Clone the repository

```
git clone https://github.com/your-repo/project-name.git

cd backend
```

Step 2: Install dependencies

```
pip install -r requirements.txt
```

Step 3: Configure environment variables

Create a .env file with the following variables:

```
DB_HOST=xxxx

DB_PORT=5432

DB_NAME=delivery_demo

DB_USER=postgres

DB_PASSWORD=password


REDIS_HOST=xxxx

REDIS_PORT=6379
```

ROUTING_ENGINE_URL=http://localhost:5000

Step 4: Initialize the database

psql -U postgres -d delivery_demo -f database/schema.sql

Step 5: Start the backend server

uvicorn api.routes:app --reload

The backend API will run at:

http://localhost:8000

Frontend Installation

Step 1: Navigate to the frontend directory

cd Frontend - Haichao

Step 2: Install project dependencies

npm install

Step 3: Configure environment variables

Create a .env file in the frontend directory.

Example:

VITE_API_BASE_URL=http://localhost:8000

VITE_AZURE_MAP_KEY=YOUR_AZURE_MAP_KEY

Step 4: Start the frontend development server

npm run dev

The frontend application will start at:

http://localhost:5173

5. Operating Instructions

Starting the System

1. Start the backend API server.
2. Start the frontend development server.
3. Open the frontend application in a web browser.

Driver Interface Overview

The Fleeton driver interface provides a map-based dashboard for delivery drivers.

The interface displays:

- current vehicle location
- delivery stops
- planned route
- real-time alerts
- suggested rerouting updates

Receiving Alerts

When the backend detects an incident that may impact delivery time, the system will display an alert notification.

Alert information includes:

- incident type
- severity level
- estimated delay impact

Updating the Route

When a new route is available, the driver can review the updated route and accept the optimized sequence.

To apply the new route:

- Review the alert notification
- Click the Update Route button
- The system updates the remaining stops and route visualization

6. Technical Specifications

System Architecture

Fleeton consists of two main components:

- Backend routing and incident detection system
- Frontend driver application

These components communicate through HTTP APIs and long-polling mechanisms.

Backend Technologies

- Core technologies used:
- Python
- FastAPI
- PostgreSQL + PostGIS
- Redis
- OSRM routing engine
- AWS Lambda
- AWS API Gateway

Key backend modules include:

- telemetry ingestion service
- event processing engine
- ETA impact analysis
- routing optimization engine
- notification service

Frontend Technologies

The driver interface is implemented using modern web technologies.

Framework:

- React
- TypeScript
- Vite

Key frontend features include:

- map visualization using Azure Maps
- real-time vehicle location display
- delivery stop list management
- alert notification interface
- dynamic route update interaction

Communication with the backend occurs through REST APIs and long-polling for event updates.

7. Security and Privacy Considerations

Fleeton processes vehicle telemetry data such as GPS location and operational status.

The prototype system does not store personal user data.

Security considerations include:

- API endpoints should be protected with authentication in production environments
- database credentials must be stored in secure environment variables
- API access should be restricted through secure gateways

Operational limitations:

- the demo system uses simulated telemetry data
- authentication and user account management are not implemented in the prototype

8. Troubleshooting

Map Not Loading

Possible causes:

- invalid Azure Maps API key
- network connection issues

Solution:

- verify API key configuration in the frontend environment file
- check internet connectivity

Backend Not Responding

Possible causes:

- backend server not running
- incorrect API endpoint configuration

Solution:

- verify backend server is running at localhost:8000
- confirm frontend environment variables are correct

No Alerts Appearing

Possible causes:

- no simulated incidents triggered
- backend event processor not active

Solution:

- verify telemetry simulation is running
- check backend logs for event processing activity

9. Limitations

Fleeton is currently a prototype system developed for demonstration and research purposes.

Known limitations include:

- incident detection is based on simplified rule-based logic
- telemetry data in the demo environment is simulated
- authentication and user management features are not implemented
- the frontend driver interface is optimized for demonstration rather than production deployment
- routing results depend on local OSRM configuration

Future improvements may include:

- machine learning based incident detection
- full mobile deployment
- secure authentication and fleet management features

Limitations

The current system has the following limitations:

- Vehicle data is simulated rather than collected from real vehicles
- The routing engine uses a simplified optimization algorithm
- Large-scale load testing has not been performed
- Production monitoring systems are not fully implemented
- Redis caching strategy is still basic

Future improvements may include:

- Integration with real vehicle telemetry devices
- Enhanced routing optimization algorithms
- Improved system scalability
- Multi-vehicle delivery optimization support